

Circuit and working

The circuit is designed using an ESP32 development board, a single-channel 5V relay module, and several jumper wires. The circuit diagram for this setup is presented in Fig. 3. The device intended for control within this setup should be connected across the relay contacts, which are not depicted in Fig. 3. Table 2 outlines the connections between the ESP32 development board and the single-channel 5V relay module.

Configuration of Arduino IDE

ESP RainMaker, an end-to-end solution by Espressif Systems, enables remote control and monitoring of ESP32-based products without requiring cloud configuration. To utilise ESP RainMaker with Arduino IDE, search for the ESP32 board in the board manager and install the latest version. In Arduino IDE, select the appropriate board, the flash size, and set the partition scheme to RainMaker. Open the code, compile it, and upload it to the ESP32 development board. Download the ESP RainMaker app and scan the QR code displayed in the serial monitor using the app.

The implementation of a node is a crucial step in this setup. A node refers to a single ESP32-based product with an associated identifier (node_id) and credentials provided by the Claiming service. Users can assign a friendly name to the node; in this setup, the node is named 'hotspot charging.' A single node can interface with multiple devices, which

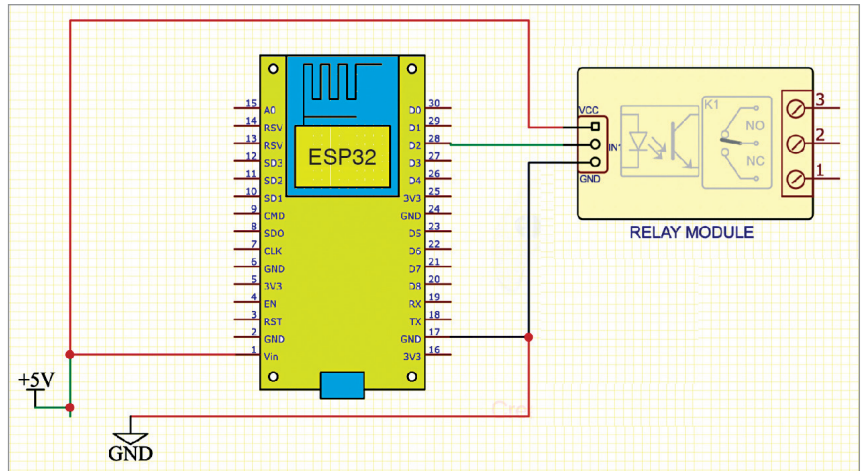


Fig. 3: Circuit diagram of the proposed setup

can be configured by users logged into the app. This node-based system operates through clients, such as a smartphone app, controlled via the ESP RainMaker cloud.

Node attributes, which are optional, serve as custom metadata, like a serial number, for a given node. A simple node typically controls a single device, such as a switch, light bulb, thermostat, or temperature sensor. In this setup, the device is the mobile hotspot/tethering unit. However, an ESP32 development board can support multiple devices, such as two switches on a single board, or act as a bridge for other devices, like Bluetooth low energy (BLE) sensors. Each device within the node must have a unique name and, optionally, a type. Fig. 4 provides a snippet of the source code used in the node implementation.

Device attributes serve as additional, user-defined, read-only name-value pairs that function as metadata for the device. Examples include firmware version, serial number, and other identifying information. These attributes are particularly valuable when multiple devices are bridged to

```
espraimmaker.ino
1  #include "RMaker.h"
2  #include "WiFi.h"
3  #include "WiFiProv.h"
4  #include "SimpleTimer.h"
5  #include <wifi_provisioning/manager.h>
6  // Set Default Values
7  #define DEFAULT_RELAY_MODE false
8  // BLE Credentials
9  const char service_name = "*****"; //name
10 const char *pop = "*****"; //password
11 // GPIO
12 static uint8_t gpio_reset = 0;
13 static uint8_t relay = 27;
14 bool relay_state = true;
15 bool wifi_connected = 0;
16 SimpleTimer Timer;
17 static Switch my_switch("Kavish", &relay);
18 void sysProvEvent(arduino_event_t *sys_event) {
```

Fig. 4: Source code snippet of the setup

a node, such as in scenarios involving Wi-Fi to wired network, BLE, or Zigbee connections. Parameters like on/off power state, brightness, and current/target temperature are employed to control and monitor the device. Each parameter must possess a unique name within the device, and they can also include additional fields such as value, type, bounds, and UI information. The values of these parameters can be expressed in boolean, integer, float, or string data types.

A service is an entity structurally similar to a device, but it is intended for operations that may not necessarily be visible to the user. A typical example is the over-the-air (OTA)

TABLE 2

CONNECTIONS BETWEEN THE ESP32 DEVELOPMENT BOARD AND RELAY MODULE

ESP32 development board	5V power supply	5V relay module
Voltage In (VIN)	+ (positive)	+ (VCC)
D2 pin	—	in (signal)
GND	- (ground)	- (GND)